

SPRING DOCS

REST API Design with Spring

Principles, HTTP Semantics, Status Codes & Best Practices

Java Agentic AI — Knowledge Base Reference

1. What Is REST?

REST (Representational State Transfer), defined by Roy Fielding in 2000, is an architectural style for networked systems, built on the existing semantics of HTTP rather than inventing a new protocol. A REST API exposes **resources** (nouns, e.g. /users, /orders) manipulated through standard HTTP methods (verbs).

Constraint	Meaning
Client-Server	Separation of concerns between UI and data storage
Stateless	Each request contains all information needed; server holds no client session state
Cacheable	Responses must define themselves as cacheable or not
Uniform Interface	Consistent resource identification (URIs) and manipulation via representations
Layered System	Client can't tell if it's talking directly to the server or through intermediaries

2. HTTP Methods and Their Semantics

Method	Purpose	Idempotent?	Safe (no side effects)?
GET	Retrieve a resource	Yes	Yes
POST	Create a new resource / trigger an action	No	No
PUT	Replace a resource entirely	Yes	No
PATCH	Partially update a resource	Not guaranteed	No
DELETE	Remove a resource	Yes	No

Note: Idempotent means calling it once has the same net effect as calling it multiple times — DELETE /users/5 called twice still results in user 5 being gone either way, so it's idempotent even though it's not 'safe' (it does change state).

3. URI Design Conventions

- Use nouns, not verbs: /users not /getUsers.
- Use plural resource names: /orders, /orders/{id}.
- Represent hierarchy with nesting: /users/{id}/orders.
- Use query parameters for filtering/sorting/pagination: /orders?status=SHIPPED&page;=2&sort;=date,desc.
- Use lowercase, hyphenated words for multi-word resources: /order-items, not /orderItems or /Order_Items.
- Version the API explicitly: /api/v1/users (URI versioning) or via an Accept header (content negotiation versioning).

4. HTTP Status Codes

Code	Meaning	Typical use
200 OK	Success	Successful GET, PUT, PATCH
201 Created	Resource created	Successful POST
204 No Content	Success, no body	Successful DELETE
400 Bad Request	Client sent invalid data	Validation failure
401 Unauthorized	Not authenticated	Missing/invalid credentials
403 Forbidden	Authenticated but not permitted	Insufficient role/permission
404 Not Found	Resource doesn't exist	Invalid ID in path
409 Conflict	State conflict	Duplicate unique field, optimistic lock failure
422 Unprocessable Entity	Semantically invalid	Well-formed but business-rule violation
500 Internal Server Error	Unexpected server fault	Uncaught exception

5. Building REST Endpoints in Spring

```
@RestController
@RequestMapping("/api/v1/orders")
public class OrderController {

    private final OrderService orderService;
    public OrderController(OrderService orderService) { this.orderService = orderService; }

    @GetMapping
    public ResponseEntity<Page<OrderDto>> getOrders(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "20") int size) {
        return ResponseEntity.ok(orderService.findAll(PageRequest.of(page, size)));
    }

    @PostMapping
    public ResponseEntity<OrderDto> create(@Valid @RequestBody CreateOrderRequest req) {
        OrderDto created = orderService.create(req);
        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
            .path("/{id}").buildAndExpand(created.getId()).toUri();
        return ResponseEntity.created(location).body(created); // 201 + Location header
    }

    @PatchMapping("/{id}")
    public ResponseEntity<OrderDto> update(@PathVariable Long id, @RequestBody Map<String,
    Object> updates) {
        return ResponseEntity.ok(orderService.partialUpdate(id, updates));
    }
}
```

6. DTOs vs Entities

Exposing JPA entities directly over the wire is discouraged: it leaks internal persistence structure, risks lazy-loading exceptions during JSON serialization, and tightly couples the API contract to the database schema. DTOs (Data Transfer Objects) decouple the two, providing control over exactly what's exposed.

```
public record UserDto(Long id, String name, String email) { }

@Component
public class UserMapper {
    public UserDto toDto(User user) {
        return new UserDto(user.getId(), user.getName(), user.getEmail());
    }
}
```

7. HATEOAS (Optional Maturity Level)

Hypermedia As The Engine Of Application State — responses include links to related/available actions, letting clients navigate the API dynamically rather than hard-coding URLs.

```
EntityModel<UserDto> resource = EntityModel.of(userDto,
    linkTo(methodOn(UserController.class).getUser(id)).withSelfRel(),
    linkTo(methodOn(UserController.class).getOrders(id)).withRel("orders"));
```

8. API Versioning Strategies

Strategy	Example	Trade-off
URI path	/api/v1/users	Simple, visible, but clutters the URI namespace
Query parameter	/api/users?version=1	Easy to default, less RESTful (versions a resource, not identifying it)
Header	Accept: application/vnd.company.v1+json	Clean URIs, but harder to test/browse manually

9. Idempotency in Practice

```
@PutMapping("/{id}")
public ResponseEntity<OrderDto> replace(@PathVariable Long id, @RequestBody OrderDto dto) {
    // Calling this 1 time or 10 times with the same body yields the identical final state
    return ResponseEntity.ok(orderService.replace(id, dto));
}
```

Note: For POST (naturally non-idempotent, e.g. payment creation), an Idempotency-Key header pattern is commonly used: the client generates a unique key per logical operation, and the server deduplicates retries carrying the same key.

10. Quick Interview Recap

- PUT vs PATCH? PUT replaces the entire resource representation; PATCH applies a partial update to specific fields.

- Why is REST called 'stateless'? The server doesn't retain any client session context between requests — every request must carry all information needed to process it (e.g. an auth token).
- 201 vs 200 vs 204? 201 = resource created (include Location header); 200 = success with a body; 204 = success with no body (common for DELETE).
- Why use DTOs instead of returning entities directly? Avoids leaking persistence internals, prevents lazy-loading serialization exceptions, and decouples the API contract from the DB schema.
- Is GET idempotent and safe? Yes to both — it doesn't change server state (safe) and repeating it yields the same result (idempotent).